AI 使用说明

用了什么，帮了什么

用了 Codex 和词元工坊（TokenWorks）里的 AI 编程助手，帮我读代码、写适配和测试脚本、修改算子、分析日志。它让我更快开始尝试，但方案是否合适、结果是否可信，需要我自己判断。

这次踩过的坑

局部对齐，容易听成整个问题解决了。

早期回复强调参考路径的样例已经对齐。回头核对同一阶段的结果表，实际运行的快路径仍没通过验收。我才意识到，“定位了原因”“改了代码”“验证通过”是不同的事，少说适用范围就会让人误解。

原生 prefill 的16K单样例完全一致，但完整C200一致率只有46.37%。我因此排除了这个候选，保留失败记录，没有把单个样例通过当作整体验收。

越聊越深入，也可能离目标越来越远。

对齐迟迟不过时，早期记录转向讨论验收指标是否合理，还尝试喂入参考答案的前缀来比较。这个方法有助于诊断，但题目要求的是自由生成结果。重新对照题目后，我意识到不能沿着 AI 的解释一直走，更不能用另一套检查代替原来的目标。

设计说得完整，前提也可能是错的。

MTP 的早期方案把注意力类型设想错了，后来对照已有实现才纠正。这个例子提醒我：写代码前要先核实模型结构，不能因为方案听起来顺，就默认它的前提成立。

我最大的收获

一个明确的目标和进展检查，比不停催 AI 赶时间更重要。我需要始终清楚：baseline 是什么，有哪些方案，现在改的是哪条路径，哪些结果真正验证过。遇到“已解决”，还要追问在哪个版本、什么场景下解决了。

AI 很适合搭框架、做 demo，也能帮我完成大量代码工作。但要把工程做成、真正学到东西，我必须亲自坐下来读代码、看日志、比较方案，想清楚为什么这样改。以后我会给自己留出深入思考的时间，不能把连续追问当成自己已经理解了。